

Experiment Results: Fine-Tuning Efficiency

Google Drive Link: <https://drive.google.com/drive/folders/1MKwgU3BGK-32tQ1DBoWvL3ZolbjLcONb?usp=sharing>

Lab 2 дээр цуглуулсан Arxiv-н өгөгдлийг ашиглан дараах туршилтуудыг гүйцэтгэлээ. Уг өгөгдөл нь судалгааны ажлуудын гарчиг болон abstract гэх мэт хувьсагчид агуулна. Иймд би Лаб 3-ын QA дээр судалгааны ажлын abstract өгөгдөхөд гарчгийг нь таамаглах модел сургахаар зорисон. Энэ хүрээнд T5 болон Parameter Efficient Fine-Tuning ашиглаж сургалт хийлээ.

Method	Model	Epochs	Trainable Params	All Params	Trainable %	ROUGE Type	Precision	Recall	F1-Score
Parameter Efficient Fine-Tuning (PEFT)									
	t5-base	5	884,736	223,788,288	0.3953	Low	0.4998	0.2737	0.3422
						Mid	0.5104	0.2805	0.3498
						High	0.5210	0.2874	0.3574
	t5-small	15	294,912	60,801,536	0.4850	Low	0.4566	0.2501	0.3121
						Mid	0.4678	0.2569	0.3197
						High	0.4782	0.2634	0.3273
	t5-efficient-mini	90	172,032	31,392,512	0.5480	Low	0.4103	0.2345	0.2883
						Mid	0.4205	0.2409	0.2955
						High	0.4310	0.2473	0.3029
	t5-efficient-mini	60	172,032	31,392,512	0.5480	Low	0.4042	0.2278	0.2809
						Mid	0.4153	0.2343	0.2889
						High	0.4262	0.2412	0.2969
	t5-efficient-mini	3	172,032	31,392,512	0.5480	Low	0.0000	0.0000	0.0000
						Mid	0.0000	0.0000	0.0000
						High	0.0000	0.0000	0.0000
Full Model Fine-Tuning	t5-small	1.5	60,801,536	60,801,536	100	Low	0.3963	0.3066	0.3338
						Mid	0.4056	0.3148	0.3421
						High	0.4146	0.3225	0.3492

Моделийн гүйцэтгэлийг жишээ болгон үзүүлэх зорилгоор тестийн өгөгдлөөс сонгон авсан дурын 7 abstract-н хувьд ажлын гарчгийг нь таамаглалаа.

T5-Base 5 epochs (PEFT):

Test index: 786

Actual Title: Generative Emotional AI for Speech Emotion Recognition: The Case for Synthetic Emotional Speech Augmentation

Predicted Title: End-to-End Text-to-

Test index: 2514

Actual Title: Nonparallel High-Quality Audio Super Resolution with Domain Adaptation and Resampling CycleGANs

Predicted Title: Dual-CycleGAN: High-

Test index: 2805

Actual Title: Inference with Deep Generative Priors in High Dimensions

Predicted Title: Multi-Layer Vector Approximate

Test index: 171

Actual Title: Spoken Language Intelligence of Large Language Models for Language Learning

Predicted Title: Multiple-choice Question Dataset: Evaluation of

Test index: 1289

Actual Title: TinyLSTMs: Efficient Neural Speech Enhancement for Hearing Aids

Predicted Title: Model compression for RNN Speech Enhancement

Test index: 1285

Actual Title: EDAC: Efficient Deployment of Audio Classification Models For COVID-19 Detection

Predicted Title: COVID-19 Pre-Screening

Test index: 2815

Actual Title: Generalization Ability of MOS Prediction Networks

Predicted Title: MOSNet and self-supervised speech models for

T5-Small 15 epochs (PEFT):

Test index: 786

Actual Title: Generative Emotional AI for Speech Emotion Recognition: The Case for Synthetic Emotional Speech Augmentation

Predicted Title: Enhancing SER with Synthetic Emotion Recognition

Test index: 2514

Actual Title: Nonparallel High-Quality Audio Super Resolution with Domain Adaptation and Resampling CycleGANs

Predicted Title: Dual-CycleGAN: High-

Test index: 2805

Actual Title: Inference with Deep Generative Priors in High Dimensions

Predicted Title: Multi-Layer Vector Approximate

Test index: 171

Actual Title: Spoken Language Intelligence of Large Language Models for Language Learning

Predicted Title: Multi-choice Question dataset for LLMs

Test index: 1289

Actual Title: TinyLSTMs: Efficient Neural Speech Enhancement for Hearing Aids

Predicted Title: Model Compression for Speech Enhancement

Test index: 1285

Actual Title: EDAC: Efficient Deployment of Audio Classification Models For COVID-19 Detection

Predicted Title: Using Machine Learning to detect COVID-19

Test index: 2815

Actual Title: Generalization Ability of MOS Prediction Networks

Predicted Title: MOSNet and Self-supervised Speech Models

T5-Efficient-Mini 60 epochs (PEFT):

Test index: 786

Actual Title: Generative Emotional AI for Speech Emotion Recognition: The Case for Synthetic Emotional Speech Augmentation

Predicted Title: A TTS system for synthetic emotional speech emotion recognition

Test index: 2514

Actual Title: Nonparallel High-Quality Audio Super Resolution with Domain Adaptation and Resampling CycleGANs

Predicted Title: Dual-CycleGAN: High-

Test index: 2805

Actual Title: Inference with Deep Generative Priors in High Dimensions

Predicted Title: Multi-Layer Vector Approximate

Test index: 171

Actual Title: Spoken Language Intelligence of Large Language Models for Language Learning

Predicted Title: LLMs: A Multi-choice Question

Test index: 1289

Actual Title: TinyLSTMs: Efficient Neural Speech Enhancement for Hearing Aids

Predicted Title: A Multi-SDR for RNN Speech Enhanc

Test index: 1285

Actual Title: EDAC: Efficient Deployment of Audio Classification Models For COVID-19 Detection

Predicted Title: COVID-19: A Using Machine Learning

Test index: 2815

Actual Title: Generalization Ability of MOS Prediction Networks

Predicted Title: A Multi-Seen Speech Model for MO

T5-Efficient-Mini 90 epochs (PEFT):

Test index: 786

Actual Title: Generative Emotional AI for Speech Emotion Recognition: The Case for Synthetic Emotional Speech Augmentation

Predicted Title: A TTS System for End-to-En

Test index: 2514

Actual Title: Nonparallel High-Quality Audio Super Resolution with Domain Adaptation and Resampling CycleGANs

Predicted Title: Dual-CycleGAN: High-

Test index: 2805

Actual Title: Inference with Deep Generative Priors in High Dimensions

Predicted Title: Multi-Layer Vector Approximate

Test index: 171

Actual Title: Spoken Language Intelligence of Large Language Models for Language Learning

Predicted Title: LLMs: A Multi-choice Question

Test index: 1289

Actual Title: TinyLSTMs: Efficient Neural Speech Enhancement for Hearing Aids

Predicted Title: A Multi-Suspecting Model for Speech

Test index: 1285

Actual Title: EDAC: Efficient Deployment of Audio Classification Models For COVID-19 Detection

Predicted Title: CoVID-19: A Streaming and

Test index: 2815

Actual Title: Generalization Ability of MOS Prediction Networks

Predicted Title: A Multi-Seen Speech Model for MO

T5-Small 1.5 epochs (Full):

Test index: 786

Actual Title: Generative Emotional AI for Speech Emotion Recognition: The Case for Synthetic Emotional Speech Augmentation

Predicted Title: SER: augmenting SER performance by End-to-End Text-to-

Test index: 2514

Actual Title: Nonparallel High-Quality Audio Super Resolution with Domain Adaptation and Resampling CycleGANs

Predicted Title: Towards Neural Audio Super-Resolution for Neural Audio Super-Resolution

Test index: 2805

Actual Title: Inference with Deep Generative Priors in High Dimensions

Predicted Title: ML-VAMP: Multi-Layer Vector Approximate Message Passing

Test index: 171

Actual Title: Spoken Language Intelligence of Large Language Models for Language Learning

Predicted Title: Towards a Multi-choice Question dataset for Language Learning

Test index: 1289

Actual Title: TinyLSTMs: Efficient Neural Speech Enhancement for Hearing Aids

Predicted Title: Adaptive Speech Enhancement for Speech Enhancement

Test index: 1285

Actual Title: EDAC: Efficient Deployment of Audio Classification Models For COVID-19 Detection

Predicted Title: Using a Network pruning and quantisation

Test index: 2815

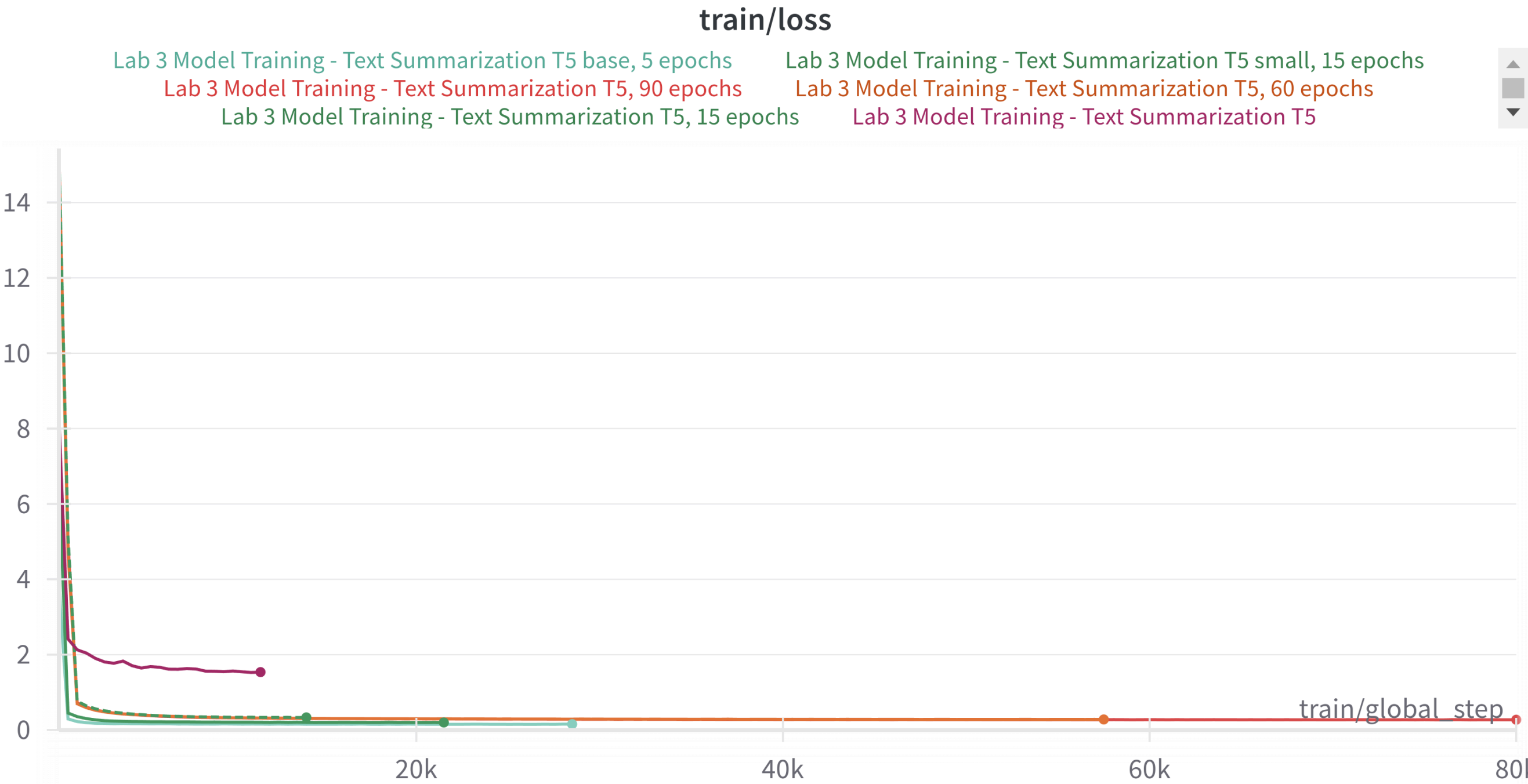
Actual Title: Generalization Ability of MOS Prediction Networks

Predicted Title: Using a variety of networks for MOS prediction models

```
In [ ]: # Сургалтын явц
from IPython.display import Image

Image(filename = "W&B Chart 3_30_2024, 5_43_11 PM.png")
```

Out[]:



```
In [ ]: import os
import pickle
from tqdm import tqdm
from glob import glob

import numpy as np
import pandas as pd
from datasets import Dataset, DatasetDict, load_metric

import torch
from transformers import (AutoTokenizer, AutoModelForSeq2SeqLM, TrainingArguments,
                          DataCollatorForSeq2Seq, Trainer, pipeline)
from peft import (LoraConfig, get_peft_model, TaskType,
                  PeftModel, PeftConfig)

import warnings
warnings.filterwarnings('ignore')

import logging
logging.basicConfig(level = logging.INFO)
transformers_logger = logging.getLogger("transformers")
transformers_logger.setLevel(logging.WARNING)
```

```
In [ ]: class CFG:
    wandb = True
    report_to = None
    lab_assignment = 3
    _wandb_kernel = "temuujin"

    debug = False
    num_workers = 12

    prefix_val = "summarize: "
    output_dir = "processed_data"
    model_save_dir = "PEFT_T5"

    tokenizer_name = "google-t5/t5-base"
    model_name = "google-t5/t5-base"

    project = 'NUM-Machine-Learning-Lab-3'
    name = "Lab 3 Model Training - Text Summarization T5 base, 5 epochs"

    config = {
        "output_dir": "t5_small_lab3_finetune_PEFT",
        "group": model_name,
        "learning_rate": 2e-5,
        "weight_decay": 1e-3,
        'num_train_epochs': 5,
        "train_batch_size": 2,
        "eval_batch_size": 8,
        "max_seq_length": 1024,
        "overwrite_output_dir": True,
        "reprocess_input_data": True,
        "fp16": True
    }

    test_size = 0.2

    train = True
```

```

eval = True

eval_metric = "rouge"

early_stopping_patience = 15

if CFG.debug:
    CFG.config['num_train_epochs'] = 2

if CFG.wandb:
    os.environ["WANDB_SILENT"] = "True"
    CFG.report_to = "wandb"

    import wandb
    wandb.login()

    run = wandb.init(
        project = CFG.project,
        name = CFG.name,
        config = CFG.config
    )

config = CFG.config

```

1. Өгөгдлөө модел сургахад бэлтгэх

```

In [ ]: df_names = glob("../Lab 2/processed_dfs/*.parquet")
search_terms = '\n'.join([os.path.basename(filename).split('_')[1] for filename in df_names])

df = pd.DataFrame()
for df_name in tqdm(df_names):
    df = pd.concat([df, pd.read_parquet(df_name)[['title', 'abstract']]])

df.drop_duplicates(inplace = True)
df.reset_index(drop = True, inplace = True)

df.rename(columns = {'title': 'target_text', 'abstract': 'input_text'}, inplace = True)
df['prefix'] = CFG.prefix_val

os.makedirs(CFG.output_dir, exist_ok = True)
output_filename = os.path.join(CFG.output_dir, "arxiv_title_generation.parquet")
df.to_parquet(output_filename)

```

100%|██████████| 12/12 [00:00<00:00, 17.01it/s]

```

In [ ]: print("\nDataframe memory usage")
print(df.memory_usage(deep = True))

print(f"Dataframe shape: {df.shape}\n")
print(f"All search terms:\n{search_terms}")

print(df.head())

```

```
Dataframe memory usage
Index          132
target_text    1966107
input_text     17616445
prefix         983416
dtype: int64
Dataframe shape: (14462, 3)
```

```
All search terms:
audio+classification
audio+deep+learning
audio+encoding
audio+fast+fourier
audio+fourier
audio+generation
audio+machine+learning
audio+prediction
audio+recognition
audio+representation
audio+restoration
audio+signal
```

```
target_text \
0 Improved Mispronunciation detection system usi...
1 Improving Factored Hybrid HMM Acoustic Modelin...
2 Disentangling Style and Speaker Attributes for...
3 Synthetic speech detection using meta-learning...
4 A Pre-trained Audio-Visual Transformer for Emo...
```

```
input_text      prefix
0 This report proposes state-of-the-art research... summarize:
1 In this work, we show that a factored hybrid h... summarize:
2 End-to-end neural TTS has shown improved perfo... summarize:
3 Recent works on speech spoofing countermeasure... summarize:
4 In this paper, we introduce a pretrained audio... summarize:
```

```
In [ ]: test_size = CFG.test_size
test_df = df.sample(frac = test_size, random_state = 1970)
train_df = df.drop(index = test_df.index)

print(f"Training instance count: {len(train_df)}\nTest instance count: {len(test_df)}\n")

train_dataset = Dataset.from_dict(train_df)
test_dataset = Dataset.from_dict(test_df)
arxiv_title_dict = DatasetDict({"train": train_dataset, "test": test_dataset})

output_filename = os.path.join(CFG.output_dir, "arxiv_title_generation_dataset")
arxiv_title_dict.save_to_disk(output_filename)

print(arxiv_title_dict)
```

```
Training instance count: 11570
Test instance count: 2892
```

```
Saving the dataset (0/1 shards): 0%|          | 0/11570 [00:00<?, ? examples/s]
Saving the dataset (0/1 shards): 0%|          | 0/2892 [00:00<?, ? examples/s]
```

```
DatasetDict({
  train: Dataset({
    features: ['target_text', 'input_text', 'prefix'],
    num_rows: 11570
  })
  test: Dataset({
    features: ['target_text', 'input_text', 'prefix'],
    num_rows: 2892
  })
})
```

```
In [ ]: tokenizer = AutoTokenizer.from_pretrained(CFG.tokenizer_name, use_fast = False)
```

```
def preprocess_function(examples):
    inputs = [CFG.prefix_val + doc for doc in examples["input_text"]]
    model_inputs = tokenizer(inputs,
                             max_length = config['max_seq_length'],
                             padding = "max_length",
                             truncation = True,
                             return_tensors = 'pt')

    labels = tokenizer(text_target = examples["target_text"],
                      max_length = config["max_seq_length"] // 4,
                      padding = "max_length",
                      truncation = True,
                      return_tensors = 'pt')

    model_inputs["labels"] = labels["input_ids"]

    return model_inputs

tokenized_arxiv = arxiv_title_dict.map(preprocess_function, batched = True)
```

You are using the default legacy behaviour of the `<class 'transformers.models.t5.tokenization_t5.T5Tokenizer'>`. This is expected, and simply means that the ``legacy`` (previous) behavior will be used so nothing changes for you. If you want to use the new behaviour, set ``legacy=False``. This should only be set if you understand what it means, and thoroughly read the reason why this was added as explained in <https://github.com/huggingface/transformers/pull/24565>

Special tokens have been added in the vocabulary, make sure the associated word embeddings are fine-tuned or trained.

```
Map:   0%|          | 0/11570 [00:00<?, ? examples/s]
```

```
Map:   0%|          | 0/2892 [00:00<?, ? examples/s]
```

Parameter Efficient Fine-Tuning (PEFT)

2.1 Model Fine-Tuning

```
In [ ]: lora_config = LoraConfig(
    task_type = TaskType.SEQ_2_SEQ_LM,
    inference_mode = False,
    r = 8,
    lora_alpha = 32,
    lora_dropout = 0.3
)

model = AutoModelForSeq2SeqLM.from_pretrained(CFG.model_name)
data_collator = DataCollatorForSeq2Seq(tokenizer = tokenizer, model = CFG.model_name)

peft_model = get_peft_model(model, lora_config)
```

```
peft_model.print_trainable_parameters()
```

```
trainable params: 884,736 || all params: 223,788,288 || trainable%: 0.3953450861557152
```

```
In [ ]: training_args = TrainingArguments(
    report_to = CFG.report_to,
    output_dir = config["output_dir"],
    num_train_epochs = config["num_train_epochs"],
    per_device_train_batch_size = config["train_batch_size"],
    overwrite_output_dir = config["overwrite_output_dir"],
    learning_rate = config["learning_rate"],
    weight_decay = config["weight_decay"],
    do_eval = False,
    disable_tqdm = False,
)

trainer = Trainer(
    model = peft_model,
    args = training_args,
    train_dataset = tokenized_arxiv["train"],
    data_collator = data_collator
)
```

```
In [ ]: torch.cuda.empty_cache()
```

```
trainer.train()
```

```
0%|          | 0/28925 [00:00<?, ?it/s]
```

```
{'loss': 3.8226, 'grad_norm': 0.7146222591400146, 'learning_rate': 1.9654278305963702e-05, 'epoch': 0.09}
{'loss': 0.2933, 'grad_norm': 0.5132521390914917, 'learning_rate': 1.93085566119274e-05, 'epoch': 0.17}
{'loss': 0.2222, 'grad_norm': 0.548663854598999, 'learning_rate': 1.89628349178911e-05, 'epoch': 0.26}
{'loss': 0.1946, 'grad_norm': 0.37290942668914795, 'learning_rate': 1.86171132238548e-05, 'epoch': 0.35}
{'loss': 0.1763, 'grad_norm': 0.5334296822547913, 'learning_rate': 1.82713915298185e-05, 'epoch': 0.43}
{'loss': 0.174, 'grad_norm': 0.3601020276546478, 'learning_rate': 1.7925669835782197e-05, 'epoch': 0.52}
{'loss': 0.1665, 'grad_norm': 0.38592833280563354, 'learning_rate': 1.7579948141745894e-05, 'epoch': 0.61}
{'loss': 0.1714, 'grad_norm': 0.4861486256122589, 'learning_rate': 1.7234226447709594e-05, 'epoch': 0.69}
{'loss': 0.1663, 'grad_norm': 0.3103805482387543, 'learning_rate': 1.6888504753673295e-05, 'epoch': 0.78}
{'loss': 0.1661, 'grad_norm': 0.37231042981147766, 'learning_rate': 1.654278305963699e-05, 'epoch': 0.86}
{'loss': 0.1636, 'grad_norm': 0.245479553937912, 'learning_rate': 1.6197061365600692e-05, 'epoch': 0.95}
{'loss': 0.1651, 'grad_norm': 0.2728257477283478, 'learning_rate': 1.5851339671564392e-05, 'epoch': 1.04}
{'loss': 0.166, 'grad_norm': 0.315685510635376, 'learning_rate': 1.5505617977528093e-05, 'epoch': 1.12}
{'loss': 0.1575, 'grad_norm': 0.44159752130508423, 'learning_rate': 1.5159896283491791e-05, 'epoch': 1.21}
{'loss': 0.1582, 'grad_norm': 0.35468849539756775, 'learning_rate': 1.481417458945549e-05, 'epoch': 1.3}
{'loss': 0.1548, 'grad_norm': 0.21478904783725739, 'learning_rate': 1.446845289541919e-05, 'epoch': 1.38}
{'loss': 0.1564, 'grad_norm': 0.33289802074432373, 'learning_rate': 1.4122731201382889e-05, 'epoch': 1.47}
{'loss': 0.1555, 'grad_norm': 0.24847497045993805, 'learning_rate': 1.3777009507346588e-05, 'epoch': 1.56}
{'loss': 0.1607, 'grad_norm': 0.2564176023006439, 'learning_rate': 1.3431287813310284e-05, 'epoch': 1.64}
{'loss': 0.1576, 'grad_norm': 0.3657689392566681, 'learning_rate': 1.3085566119273985e-05, 'epoch': 1.73}
{'loss': 0.1539, 'grad_norm': 0.3013959527015686, 'learning_rate': 1.2739844425237683e-05, 'epoch': 1.82}
{'loss': 0.1562, 'grad_norm': 0.42076370120048523, 'learning_rate': 1.2394122731201384e-05, 'epoch': 1.9}
{'loss': 0.1557, 'grad_norm': 0.3209218680858612, 'learning_rate': 1.2048401037165082e-05, 'epoch': 1.99}
{'loss': 0.152, 'grad_norm': 0.31283265352249146, 'learning_rate': 1.1702679343128783e-05, 'epoch': 2.07}
{'loss': 0.1573, 'grad_norm': 0.29315510392189026, 'learning_rate': 1.1356957649092481e-05, 'epoch': 2.16}
{'loss': 0.1522, 'grad_norm': 0.3511327803134918, 'learning_rate': 1.101123595505618e-05, 'epoch': 2.25}
{'loss': 0.1561, 'grad_norm': 0.2918175160884857, 'learning_rate': 1.066551426101988e-05, 'epoch': 2.33}
{'loss': 0.1509, 'grad_norm': 0.4207204282283783, 'learning_rate': 1.0319792566983579e-05, 'epoch': 2.42}
{'loss': 0.1527, 'grad_norm': 0.45526283979415894, 'learning_rate': 9.974070872947278e-06, 'epoch': 2.51}
{'loss': 0.1522, 'grad_norm': 0.3067965805530548, 'learning_rate': 9.628349178910978e-06, 'epoch': 2.59}
{'loss': 0.1511, 'grad_norm': 0.2970256805419922, 'learning_rate': 9.282627484874677e-06, 'epoch': 2.68}
{'loss': 0.1548, 'grad_norm': 0.396047443151474, 'learning_rate': 8.936905790838375e-06, 'epoch': 2.77}
{'loss': 0.1521, 'grad_norm': 0.30446526408195496, 'learning_rate': 8.591184096802076e-06, 'epoch': 2.85}
{'loss': 0.1577, 'grad_norm': 0.33968088030815125, 'learning_rate': 8.245462402765774e-06, 'epoch': 2.94}
{'loss': 0.1506, 'grad_norm': 0.32880544662475586, 'learning_rate': 7.899740708729473e-06, 'epoch': 3.03}
{'loss': 0.149, 'grad_norm': 0.35961976647377014, 'learning_rate': 7.5540190146931725e-06, 'epoch': 3.11}
{'loss': 0.1538, 'grad_norm': 0.39901313185691833, 'learning_rate': 7.208297320656872e-06, 'epoch': 3.2}
{'loss': 0.1497, 'grad_norm': 0.2275342047214508, 'learning_rate': 6.8625756266205715e-06, 'epoch': 3.28}
{'loss': 0.1549, 'grad_norm': 0.22860848903656006, 'learning_rate': 6.51685393258427e-06, 'epoch': 3.37}
{'loss': 0.1532, 'grad_norm': 0.546814501285553, 'learning_rate': 6.17113223854797e-06, 'epoch': 3.46}
{'loss': 0.1509, 'grad_norm': 0.31276071071624756, 'learning_rate': 5.825410544511668e-06, 'epoch': 3.54}
{'loss': 0.1499, 'grad_norm': 0.35201722383499146, 'learning_rate': 5.479688850475368e-06, 'epoch': 3.63}
{'loss': 0.1497, 'grad_norm': 0.35000869631767273, 'learning_rate': 5.133967156439067e-06, 'epoch': 3.72}
{'loss': 0.15, 'grad_norm': 0.24109941720962524, 'learning_rate': 4.788245462402766e-06, 'epoch': 3.8}
{'loss': 0.1506, 'grad_norm': 0.37547269463539124, 'learning_rate': 4.4425237683664655e-06, 'epoch': 3.89}
{'loss': 0.1529, 'grad_norm': 0.37012532353401184, 'learning_rate': 4.096802074330164e-06, 'epoch': 3.98}
{'loss': 0.1534, 'grad_norm': 0.3194848299026489, 'learning_rate': 3.751080380293864e-06, 'epoch': 4.06}
{'loss': 0.1506, 'grad_norm': 0.24645495414733887, 'learning_rate': 3.4053586862575627e-06, 'epoch': 4.15}
{'loss': 0.1508, 'grad_norm': 0.46009138226509094, 'learning_rate': 3.059636992221262e-06, 'epoch': 4.24}
{'loss': 0.1489, 'grad_norm': 0.2736685872077942, 'learning_rate': 2.7139152981849613e-06, 'epoch': 4.32}
{'loss': 0.1499, 'grad_norm': 0.18686240911483765, 'learning_rate': 2.3681936041486608e-06, 'epoch': 4.41}
{'loss': 0.1491, 'grad_norm': 0.26044827699661255, 'learning_rate': 2.02247191011236e-06, 'epoch': 4.49}
{'loss': 0.149, 'grad_norm': 0.3125161826610565, 'learning_rate': 1.6767502160760591e-06, 'epoch': 4.58}
{'loss': 0.1523, 'grad_norm': 0.21138925850391388, 'learning_rate': 1.3310285220397582e-06, 'epoch': 4.67}
{'loss': 0.1496, 'grad_norm': 0.38649284839630127, 'learning_rate': 9.853068280034573e-07, 'epoch': 4.75}
{'loss': 0.1555, 'grad_norm': 0.4241676330566406, 'learning_rate': 6.395851339671565e-07, 'epoch': 4.84}
{'loss': 0.154, 'grad_norm': 0.3399752676486969, 'learning_rate': 2.938634399308557e-07, 'epoch': 4.93}
{'train_runtime': 5984.8116, 'train_samples_per_second': 9.666, 'train_steps_per_second': 4.833, 'train_loss': 0.22313696107064843, 'epoch': 5.0}
```

```
Out[ ]: TrainOutput(global_step=28925, training_loss=0.22313696107064843, metrics={'train_runtime': 5984.8116, 'train_samples_per_second': 9.666, 'train_steps_per_second': 4.833, 'train_loss': 0.22313696107064843, 'epoch': 5.0})
```

```
In [ ]: test_ver = 10
model_save_path = os.path.join(CFG.model_save_dir, f"test_v{test_ver}")
os.makedirs(model_save_path, exist_ok = True)

trainer.model.save_pretrained(model_save_path)
tokenizer.save_pretrained(model_save_path)

print(f"Saved model to: {model_save_path}")
```

Saved model to: PEFT_T5/test_v10

2.2 Evaluation

```
In [ ]: rouge = load_metric(CFG.eval_metric)

def compute_metrics(decoded_preds, decoded_labels):
    rouge_scores = rouge.compute(predictions = decoded_preds,
                                references = decoded_labels,
                                rouge_types = ["rouge1"])["rouge1"]

    return rouge_scores

def print_rouge_scores(model_results):
    rouge1_low = model_results.low
    rouge1_mid = model_results.mid
    rouge1_high = model_results.high

    print("| ROUGE Type      | Precision | Recall   | F1-Score |")
    print("-" * 54)

    print("| ROUGE-1 (Low ) | {:.4f}    | {:.4f}   | {:.4f}   |".format(rouge1_low.precision, rouge1_low.recall, rouge1_low.fmeasure))
    print("| ROUGE-1 (Mid ) | {:.4f}    | {:.4f}   | {:.4f}   |".format(rouge1_mid.precision, rouge1_mid.recall, rouge1_mid.fmeasure))
    print("| ROUGE-1 (High) | {:.4f}    | {:.4f}   | {:.4f}   |".format(rouge1_high.precision, rouge1_high.recall, rouge1_high.fmeasure))
```

```
In [ ]: peft_model_id = model_save_path
config = PeftConfig.from_pretrained(peft_model_id)

model = AutoModelForSeq2SeqLM.from_pretrained(config.base_model_name_or_path)
model = PeftModel.from_pretrained(model, peft_model_id)

tokenizer = AutoTokenizer.from_pretrained(config.base_model_name_or_path)
```

```
In [ ]: torch.cuda.empty_cache()

device = 'cuda'

model = model.to(device)
model.eval()

batch_size = 16

with torch.no_grad():
    all_decoded_results = []

    for i in tqdm(range(0, len(tokenized_arxiv['test']['input_ids']), batch_size),
                  total = len(tokenized_arxiv['test']['input_ids']) // batch_size + 1):
        batch_input_ids = torch.tensor(tokenized_arxiv['test']['input_ids'][i:i+batch_size]).to(device)
```

```

        batch_encoded_results = model.generate(input_ids = batch_input_ids,
                                                max_new_tokens = 10)
        batch_decoded_results = tokenizer.batch_decode(batch_encoded_results.detach().cpu().numpy(),
                                                        skip_special_tokens = True)
        all_decoded_results.extend(batch_decoded_results)

file_path = "./PEFT_T5/test_v7/all_decoded_results.pkl"
with open(file_path, "wb") as file:
    pickle.dump(all_decoded_results, file)
print(f"Decoded predictions saved to: {file_path}")

```

```

100%|██████████| 181/181 [05:37<00:00, 1.87s/it]
Decoded predictions saved to: ./PEFT_T5/test_v7/all_decoded_results.pkl

```

```

In [ ]: model_results = compute_metrics(all_decoded_results, tokenized_arxiv['test']['target_text'])
        print_rouge_scores(model_results)

```

```
INFO:absl:Using default tokenizer.
```

ROUGE Type	Precision	Recall	F1-Score
ROUGE-1 (Low)	0.4998	0.2737	0.3422
ROUGE-1 (Mid)	0.5104	0.2805	0.3498
ROUGE-1 (High)	0.5210	0.2874	0.3574

2.3 Checking out some predictions

```

In [ ]: def print_random_samples(all_decoded_results, target_texts, num_samples = 9):
        rand_indices = np.random.randint(1, tokenized_arxiv['test'].num_rows, size = num_samples)
        rand_pred_title = np.asarray(all_decoded_results)[rand_indices]
        rand_target_title = np.asarray(target_texts)[rand_indices]

        for idx, (predicted_title, actual_title) in enumerate(zip(rand_pred_title, rand_target_title)):
            print(f"Test index: {rand_indices[idx]}")
            print("Predicted Title:", predicted_title)
            print("Actual Title:", actual_title)
            print("-" * 64)

        print_random_samples(all_decoded_results, tokenized_arxiv['test']['target_text'], num_samples = 3)

```

```

Test index: 861
Predicted Title: Detecting Spoof Speech and Identifying
Actual Title: Evince the artifacts of Spoof Speech by blending Vocal Tract and Voice Source Features
-----
Test index: 1295
Predicted Title: Automated Recognition and Diagnostic Tool for Screening
Actual Title: Can Machine Learning Be Used to Recognize and Diagnose Coughs?
-----
Test index: 1131
Predicted Title: A Comparison of Deep Learning Models on the DC
Actual Title: A Comparison of deep learning methods for environmental sound
-----

```

Full Model Fine-Tuning: T5 small

3.1 Loading already finetuned model

```
In [ ]: full_finetuned_model = pipeline("summarization", model = "./t5_small_lab3_finetune/checkpoint-11500", batch_size = batch_size)
```

Special tokens have been added in the vocabulary, make sure the associated word embeddings are fine-tuned or trained.
You set `add\_prefix\_space`. The tokenizer needs to be converted from the slow tokenizers
You set `add\_prefix\_space`. The tokenizer needs to be converted from the slow tokenizers

3.2 Full model evaluation

```
In [ ]: torch.cuda.empty_cache()

with torch.no_grad():
    all_decoded_results = []

    for i in tqdm(range(0, len(tokenized_arxiv['test']['input_ids']), batch_size),
                  total = len(tokenized_arxiv['test']['input_ids']) // batch_size + 1):
        batch_input = [CFG.prefix_val + input_text for input_text in tokenized_arxiv['test']['input_text'][i:i+batch_size]]
        pred = full_finetuned_model(batch_input)
        pred = [pred_text['summary_text'] for pred_text in pred]

        all_decoded_results.extend(pred)

file_path = "./PEFT_T5/test_v7/full_model_finetune_test_all_decoded_results.pkl"
with open(file_path, "wb") as file:
    pickle.dump(all_decoded_results, file)
print(f"Decoded predictions saved to: {file_path}")
```

100%|██████████| 181/181 [04:34<00:00, 1.52s/it]
Decoded predictions saved to: ./PEFT_T5/test_v7/full_model_finetune_test_all_decoded_results.pkl

```
In [ ]: model_results = compute_metrics(all_decoded_results, tokenized_arxiv['test']['target_text'])
print_rouge_scores(model_results)
```

INFO:absl:Using default tokenizer.

ROUGE Type	Precision	Recall	F1-Score
ROUGE-1 (Low)	0.3963	0.3066	0.3338
ROUGE-1 (Mid)	0.4056	0.3148	0.3421
ROUGE-1 (High)	0.4146	0.3225	0.3492

```
In [ ]: print_random_samples(all_decoded_results, tokenized_arxiv['test']['target_text'], num_samples = 3)
```

Test index: 1096
Predicted Title: Using Unlabeled Data for Unlabeled Video Data
Actual Title: Evolving Losses for Unlabeled Video Representation Learning

Test index: 1639
Predicted Title: a temporal-spatial neural filter for Target Speech Separation
Actual Title: Temporal-Spatial Neural Filter: Direction Informed End-to-End Multi-channel Target Speech Separation

Test index: 2170
Predicted Title: XXX: A novel system for Speech Recognition
Actual Title: Lightweight Protection for Privacy in Offloaded Speech Understanding

```
In [ ]:
```